

QuoteGuard — Step-by-Step Build Guide

Format throughout: Why (business reason) → What (logic/output) → How (mechanical build steps and key formula patterns).

Part A — Lookup_Tables (single source of truth)

Why: Every price, rate, and assumption used anywhere in the workbook must live in exactly one place — otherwise the model can silently disagree with itself the moment one number changes.

What it contains (A.1–A.9):

- A.1 Customer size bands, defined by licensed user count (not employee headcount, which is only a loose proxy) — Small/Mid/Enterprise/Strategic — plus a Boundary Review Threshold (5%) so a deal near a band edge gets flagged for manual review instead of silently auto-assigned.
- A.2 Core Platform base price matrix (Essentials/Advanced × 4 bands).
- A.3 Add-On uplift % table — MDDR, Identity Resolution, Identity Posture, AI Security (Atlas) — each priced as a % of Core Platform, because they extend the platform rather than standing alone.
- A.4 Additional Modules flat-price table — 7 standalone products (Database Activity Monitoring, Data-Centric UEBA, DLP, ITDR, Interceptor Email Security, Interceptor Browser Security, Email Data Protection) — plus a scope note explaining why Varonis' "Solutions" and industry pages are deliberately not priced here (they're bundles of what's already priced, not separate SKUs).
- A.5 Deployment (Cloud/On-Prem) and contract term (1/2/3yr) adjustment factors.
- A.6 Implementation Fee table, T-shirt sized, one row per licensable component — Core Platform (split by deployment type) plus every Add-On and Additional Module.
- A.7 Industry reference table — informational only, drives a recommendation note in Deal_Input, never touches a price cell (keeps industry out of the pricing formula to avoid any appearance of arbitrary/discriminatory pricing).
- A.8 Renewal Uplift % assumption (7%, mid-point of a typical 5–10% enterprise SaaS range).
- A.9 Currency & FX reference rates (USD/GBP/EUR) — defaults only; each calculation sheet has its own override.

How: Every price is a manually-entered, clearly-labeled [ASSUMPTION] cell (yellow highlight) since none of these figures are publicly disclosed by the real vendor — each is anchored to a public reference point where one exists (e.g., Core Platform pricing anchored to a ~\$58K average-contract-value data point).

Part B — Deal_Input (new-business form)

Why: One clean form an account manager fills out, with guardrails against bad data.

What/How:

- Licensed User Count and Number of Data Sources as raw inputs; Assigned Size Band is a formula, not a dropdown — computed via a nested IF chain checking the count against Lookup_Tables' band thresholds, so no one can manually mismatch band and user count.
- Boundary Review Flag — compares the user count's distance to the nearest band threshold as a %, flags if within 5%.
- Deployment/Term/Tier/Industry dropdowns (Data Validation, sourced from Lookup_Tables where applicable so the list never needs retyping).
- Industry selection drives an informational "Recommended Add-Ons" note via INDEX/MATCH — never a price change.
- Yes/No dropdowns for all 4 Add-Ons and all 7 Additional Modules.

- Account Manager Name and Team/Market Segment — free-text input cells (not a locked roster dropdown), so any AM can use the tool without being pre-registered.

Part C — Pricing_Calculation (hidden engine)

Why: This is the actual pricing logic — kept hidden and password-protected so account managers only ever see outputs, not the underlying uplift %s or T-shirt tables.

What/How — the calculation chain:

1. Core Platform Base Price — two-dimensional INDEX/MATCH (tier × band).
2. Each Add-On's Amount = Core Base × its uplift %, gated by IF(Selected="Yes", ..., 0) .
3. Each Additional Module's Amount = its own flat unit price (by band), same Yes/No gate.
4. Platform Subtotal = Core + all Add-Ons + all Additional Modules — every module (including what used to be a special-cased "Interceptor" product) flows through one generic loop, which simplified the whole downstream chain considerably.
5. Cloud/On-Prem multiplier and Term factor applied in sequence → TOTAL LIST ACV (USD).
6. Implementation Fee block — same per-component pattern as pricing, summed into TOTAL IMPLEMENTATION FEE (USD), kept structurally separate from the recurring total since it's one-time and must never be multiplied by the term factor.
7. Currency & FX section — pulls the Selected Currency from Deal_Overview, looks up the Default FX Rate from Lookup_Tables, and provides a Manual FX Rate Override cell (blank = use default) so the pricing team can adjust rates on a live-business-guidance basis without editing the reference table itself. Effective FX Rate = IF(override blank, default, override) .
8. Approval Workflow table — 3 rows (Pricing Analyst / Manager Approved / Final Quote Sent), each a dropdown defaulting to "Needs Review," with real Excel conditional formatting (CellsRule) turning the cell green when "Approved" and red otherwise — dynamic, not just generated-once styling.

Part D — Deal_Overview (AM-facing summary)

Why: The single screen an account manager actually works from — needs to separate recurring from one-time, show a live discount simulator, and never expose internal pricing mechanics.

What/How — layout, top to bottom:

- Currency dropdown (USD/GBP/EUR) at the very top — every dollar figure below is converted live via the Effective FX Rate pulled from Pricing_Calculation.
- Deal Snapshot — Deal Name, AM Name, Team, Users, Band, Deployment, Term, Tier, Industry, all linked from Deal_Input.
- RECURRING (ARR) — full line-item breakdown: Core Platform, each of the 4 Add-Ons, each of the 7 Additional Modules, then Concierge/Athena AI as static "Included" cells — then List ACV, a live Discount % input cell, Net ACV, PUPY, PUPM.
- ONE-TIME (Implementation) — the same full itemization (Core Platform + 4 Add-Ons + 7 Modules), each showing its actual dollar figure (including \$0 for unselected items, so the breakdown is complete and auditable), then Total Implementation Fee, with the mandatory "Indicative estimate" caption directly beneath.
- Contract Value & Metrics — TCV, Effective Discount %, Implementation-as-%-of-Year-1-ACV, and a single linked Competitor Delta cell.
- Approval Workflow status — read-only mirror of Pricing_Calculation's 3-row table, same green/red conditional formatting, so an AM can see quote status without ever touching the protected sheet.

Part E — Competitive_Benchmark (isolated, scope-aware)

Why: Competitor pricing is manually researched, not derivable — and a fixed dollar quote goes stale the instant a deal's module selection changes, making any "benchmark" wrong within one edit.

What: Competitor pricing modeled as a positioning multiplier against Varonis' own List ACV for this specific deal (e.g., Microsoft Purview ≈0.70×, reflecting M365-bundling advantage but narrower breadth), rather than a static number — so the comparison automatically stays proportional as the quote's scope changes.

How: One-way bridges only — Competitive_Benchmark pulls Varonis' List ACV in once; a single Delta cell flows back out to Deal_Overview. Nothing else crosses between the two sheets, protecting both the pricing engine and the competitive research from accidentally corrupting each other.

Part F — Renewal_Input / Renewal_Calculation / Renewal_Overview

Why: Renewals are a core part of this role and mechanically different from new business — they start from the prior contract value, not from Lookup_Tables fresh, and carry their own governance risks (shrinkage, migration).

What/How:

- Renewal_Input — Prior Contract Value/Tier/Deployment, current user count, prior status vs. requested action (No Change/Add/Remove) for every Add-On and Additional Module, Deployment Change Requested (No/On-Prem→Cloud/Cloud→On-Prem), free-text AM fields.
- Renewal_Calculation (hidden): Base Renewal Value = Prior Value × (1 + Renewal Uplift %).
- Add/Remove adjustments — a guarded formula (IF(AND(Requested="Add", Prior="No"), ..., IF(AND(Requested="Remove", Prior="Yes"), -..., 0))) so a request only fires if it represents genuine change, never double-charging an already-active item.
- Deployment Migration block — derives the new deployment type, computes a Migration Recurring Adjustment (reprices the base value at the new deployment multiplier — this is what models the real risk of on-prem→cloud pricing coming in materially higher at renewal), a Migration Implementation Fee (reusing the same lookup pattern as new business), and an independent Migration Escalation Flag.
- Newly Added Module Implementation Fees — any Add-On/Module transitioning No→Add during the renewal correctly triggers its own implementation fee, itemized per component, summed into a combined Total Implementation Fee alongside the migration figure.
- Renewal Shrinkage Flag — fires independently of the migration flag whenever Final ACV falls below Prior Contract Value, regardless of whether a discount was even requested — because a shrinking renewal signals churn risk, a fundamentally different governance question than "how big a discount."
- Same Currency/FX and Approval Workflow pattern as Pricing_Calculation.
- Renewal_Overview (AM-facing) — mirrors Deal_Overview's structure: Currency dropdown, snapshot, itemized recurring breakdown (Base Renewal Value shown without exposing the uplift % itself, then each module's dollar adjustment), itemized implementation breakdown (Core Platform migration line + any newly-added module lines), Growth/Shrinkage and Migration flags, Renewal TCV, and the same read-only Approval Workflow mirror.

Part G — Deal_Log (reporting table)

Why: Every other sheet quotes one deal at a time. Any future reporting or visualization work needs many rows, one per deal.

What/How:

- What/How: ~18 synthetic deals, each row computing List ACV/PUPY/Implementation/TCV live via the same INDEX/MATCH and nested-IF patterns used in Pricing_Calculation (so changing a Lookup_Tables price recalculates every row here too), with randomly-generated AM names/teams, deal dates, discount %, and Shrinkage/Migration/Approval flags for variety.

- Clearly labeled as illustrative — in real use, each completed and approved quote's key figures would be pasted-as-values here as a permanent record.
- Built deliberately as a flat, row-per-deal table — no restructuring would be needed to point Power BI or Excel PivotTables/Charts at it later.

Part H — Hide & Protect

Why: Account managers should only ever see the 6 working sheets (Deal_Input, Deal_Overview, Renewal_Input, Renewal_Overview, Competitive_Benchmark, Deal_Log) — never the pricing mechanics.

How: Lookup_Tables , Pricing_Calculation , and Renewal_Calculation are set to hidden (sheet_state = "hidden") and password-protected (sheet.protection), with the workbook's structure itself locked (WorkbookProtection(lockStructure=True)) so the hidden sheets can't even be unhidden without the password. This is basic Excel-level protection — enough to keep the tool clean for its intended audience, not a substitute for enterprise access control.

Build Sequence Summary

1. Lookup_Tables (Part A)
2. Deal_Input → Pricing_Calculation (Parts B–C, interdependent — build together)
3. Deal_Overview → Competitive_Benchmark (Parts D–E)
4. Renewal_Input → Renewal_Calculation → Renewal_Overview (Part F, in that order)
5. Deal_Log (Part G) — last, since it references the same lookup logic as everything above
6. Hide & Protect (Part H) — final step, once every formula has been verified

Roadmap — Visualization Layer (Not Yet Built)

Deal_Log 's flat, formula-driven structure was built specifically so it could feed a BI tool without rework. The intended design, for reference:

- 4 report pages, matched to the three altitudes of a typical deal-desk reporting chain (Analyst → Manager → Leadership): a daily Command Center (open queue, aging, flagged deals), a Pricing Governance view (blended discount rate by account manager, discount-vs-deal-size patterns), a Pipeline & Mix view (new vs. renewal split, module attach rates, currency exposure), and a Renewal Health view (shrinkage trend, migration-flagged renewals).
- Headline metrics use a weighted blended rate, never a naive average across deals — e.g., Blended Discount % = $1 - \text{DIVIDE}([\text{Total Net ACV}], [\text{Total List ACV}])$ rather than averaging the discount % column directly, which would let a small deal's discount count as much as a large one's.

This layer is scoped but not built — a natural next step if there's reason to revisit the project.